

Final Solution

Q1.

- a. Artificial Intelligence (AI) means making computers or machines think and act like humans.

AI systems can:

- Think (analyze problems)
- Learn (from data or experience)
- Decide (choose the best action)
- Act (perform tasks such as speaking, driving, or recognizing faces)

The Turing Test checks if a computer can fool a human into thinking it's also human during a conversation.

If the human can't tell which is the computer, the machine passes the test.

- b. An agent is anything that can perceive its environment and act on it.

Core Role: Bridge between environment and intelligent behaviour.

Environment → [Agent box] → *Environment*

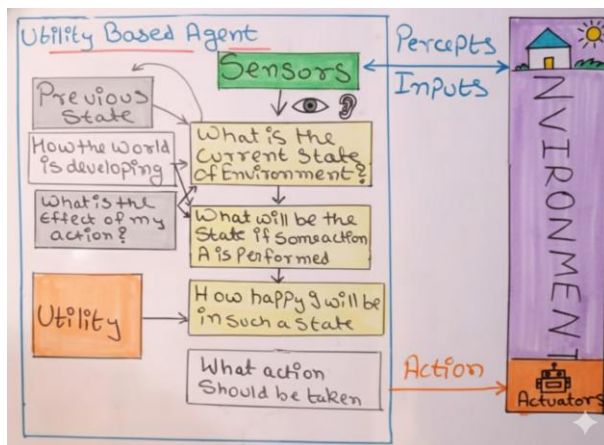
The agent gets input (percepts) from its environment and takes actions that affect it.

Perception → Decision → Action → Environment

Example:

A cleaning robot sees dirt (perception), moves to it (action).

c.



In Artificial Intelligence, a Utility-Based Agent is an intelligent agent that chooses actions based on a utility function to achieve the most desirable outcome.

OR

A Utility-Based Agent is an AI agent that selects actions by evaluating possible outcomes using a utility function and choosing the action that maximizes utility.

Key Idea:

- Compares different outcomes
- Measures how good or desirable each state is
- Chooses the action with the highest utility

Example:

A self-driving car may choose a route that balances short distance, safety, and fuel efficiency to maximize overall utility.

- d. The environment in AI is the world where an agent operates, perceives information through sensors, and performs actions using actuators.

Types of AI Environments (Any Three)

1. Fully Observable Environment

In this environment, the agent can observe the complete state of the environment at all times.

Example:

Chess game (all pieces are visible).

2. Partially Observable Environment

In this environment, the agent cannot see the complete state of the environment, so it must make decisions with limited information.

Example:

Driving a car in fog.

3. Deterministic Environment

In a deterministic environment, the next state is completely determined by the current state and the agent's action. There is no randomness.

Example:

Solving a mathematical puzzle.

Q2:

a) Informed Search:

Informed search uses a heuristic to estimate the cost from a node to the goal, guiding the search toward the most promising paths and reducing unnecessary exploration.

b) Greedy Best-First Search:

Greedy Best-First Search always expands the node with the lowest heuristic value, making it fast, but it may fail to find the optimal path.

c) A* Search:

A* search selects nodes based on $f(n) = g(n) + h(n)$, combining actual path cost and heuristic estimate to find an optimal path efficiently if the heuristic is admissible.

d) We have:

- Start node: A
 - Goal node: G
 - Heuristic $h(n)$ provided in the table
 - Edge costs are labeled on the graph
-

1 Greedy Best-First Search (GBFS)

Rule: Always expand the node with the lowest heuristic $h(n)$. Ignore path cost so far.

Step-by-step:

1. Start at A.
 - Neighbors: B ($h=12$), C ($h=11$)
 - Choose C (lowest $h=11$)
2. Expand C.
 - Neighbors: D ($h=6$), E ($h=4$)
 - Choose E (lowest $h=4$)
3. Expand E.
 - Neighbors: G ($h=0$)
 - Choose G

Path (GBFS): $A \rightarrow C \rightarrow E \rightarrow G$

Note: Total cost = 3 ($A \rightarrow C$) + 10 ($C \rightarrow E$) + 5 ($E \rightarrow G$) = 18

2 A Search*

Rule: Expand node with lowest $f(n) = g(n) + h(n)$, where

- $g(n)$ = path cost from start to node n
- $h(n)$ = heuristic estimate to goal

Step-by-step:

1. Start at A
 - $g(A)=0, h(A)=14 \rightarrow f(A)=0+14=14$
 - Neighbors:
 - B: $g(B)=4, h(B)=12 \rightarrow f(B)=4+12=16$
 - C: $g(C)=3, h(C)=11 \rightarrow f(C)=3+11=14$
 - Choose C (lowest $f=14$)
2. Expand C
 - $g(C)=3$
 - Neighbors:
 - D: $g(D)=3+7=10, h(D)=6 \rightarrow f(D)=16$
 - E: $g(E)=3+10=13, h(E)=4 \rightarrow f(E)=17$
 - Choose D (lowest $f=16$)
3. Expand D
 - $g(D)=10$
 - Neighbor: E: $g(E)=10+2=12, h(E)=4 \rightarrow f(E)=12+4=16$
 - Now E has $f(E)=16$ (from D) vs $f(E)=17$ (from C path). Take lower f , so $g(E)=12$
4. Expand E
 - $g(E)=12$
 - Neighbor: G: $g(G)=12+5=17, h(G)=0 \rightarrow f(G)=17+0=17$
5. Expand G $\rightarrow$ goal reached

Path (A*): $A \rightarrow C \rightarrow D \rightarrow E \rightarrow G$

Total cost: $3 (A \rightarrow C) + 7 (C \rightarrow D) + 2 (D \rightarrow E) + 5 (E \rightarrow G) = 17$  Optimal

Algorithm	Path	Total Cost	Notes
GBFS	$A \rightarrow C \rightarrow E \rightarrow G$	18	Fast, may not be optimal
A*	$A \rightarrow C \rightarrow D \rightarrow E \rightarrow G$	17	Optimal path guaranteed

Q3:

- a) Machine Learning is a type of AI where computers learn patterns from data to make predictions or decisions without being explicitly programmed. Unlike traditional programming, which uses fixed rules to produce output, ML learns the rules from examples:

Traditional Programming: Data + Program → Output

Machine Learning: Data + Output → Program

It's basically letting the computer figure out *how to do something* instead of telling it step by step.

- b) Supervised Learning is a type of machine learning where the model is trained on labeled data-input data with known outputs-so it can predict outputs for new inputs.

Unsupervised Learning is a type of machine learning where the model works on unlabeled data, finding patterns, relationships, or groupings without predefined outputs.

Supervised = learns from examples with answers

Unsupervised = finds patterns without answers.

- c) K-Nearest Neighbor (KNN) is a simple algorithm that classifies a new data point based on the majority class of its 'k' closest neighbors in the training data. It uses distance (like Euclidean) to find these neighbors.

- d) Training Data:

Point	X	Y	Class
-------	---	---	-------

A	2	4	Red
---	---	---	-----

B	4	6	Red
---	---	---	-----

C	6	8	Blue
---	---	---	------

D	8	10	Blue
---	---	----	------

New Point:

P = (5, 6)

Step 1: Calculate Euclidean distances from P to all training points

$$\text{Distance}(P, A) = \sqrt{(5-2)^2 + (6-4)^2} = \sqrt{9 + 4} = \sqrt{13} \approx 3.61$$

$$\text{Distance}(P, B) = \sqrt{(5-4)^2 + (6-6)^2} = \sqrt{1 + 0} = \sqrt{1} = 1$$

$$\text{Distance}(P, C) = \sqrt{(5-6)^2 + (6-8)^2} = \sqrt{1 + 4} = \sqrt{5} \approx 2.24$$

$$\text{Distance}(P, D) = \sqrt{(5-8)^2 + (6-10)^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

Step 2: Sort distances and select K=3 nearest neighbors

Nearest 3 neighbors:

B → Distance = 1, Class = Red

C → Distance = 2.24, Class = Blue

A → Distance = 3.61, Class = Red

Step 3: Majority vote among neighbors

Red → 2 votes (B, A)

Blue → 1 vote (C)

Step 4: Classify P

P is classified as: Red

Q4:

a)

A Confusion Matrix is a table used in machine learning to evaluate the performance of a classification model. It compares the predicted classes with the actual classes, showing how many predictions were correct or incorrect for each class.

It contains four main terms for binary classification:

- True Positive (TP): Correctly predicted positive
- True Negative (TN): Correctly predicted negative
- False Positive (FP): Incorrectly predicted positive
- False Negative (FN): Incorrectly predicted negative

Importance:

- Helps measure accuracy, precision, recall, and F1-score

- Shows which classes are being misclassified
- Provides deeper insight than overall accuracy alone

b)

- TP (True Positive) = 50
- FN (False Negative) = 5
- FP (False Positive) = 10
- TN (True Negative) = 35

Formulas

1. Accuracy = $\frac{TP+TN}{TP+TN+FP+FN}$
 2. Sensitivity (Recall / True Positive Rate) = $\frac{TP}{TP+FN}$
 3. Specificity (True Negative Rate) = $\frac{TN}{TN+FP}$
-

Calculations

1. Accuracy:

$$\frac{TP + TN}{TP + TN + FP + FN} = \frac{50 + 35}{50 + 35 + 5 + 10} = \frac{85}{100} = 0.85 \text{ (85\%)}$$

2. Sensitivity:

$$\frac{TP}{TP + FN} = \frac{50}{50 + 5} = 0.9 \text{ (90\%)}$$

3. Specificity:

$$\frac{TN}{TN + FP} = \frac{35}{35 + 10} = 0.77 \text{ (77\%)}$$

- c) A Bayesian Network is a graphical model that represents a set of variables and their probabilistic dependencies using a directed acyclic graph (DAG). Each node represents a variable, and edges represent conditional dependencies between them.

d)

Bayesian Network Problem: Wet Grass

Given:

$$P(R) = 0.3 \quad \Rightarrow P(\neg R) = 0.7$$

$$P(S|R) = 0.01 \quad \Rightarrow P(\neg S|R) = 0.99$$

$$P(S|\neg R) = 0.40 \quad \Rightarrow P(\neg S|\neg R) = 0.60$$

Conditional probabilities for wet grass:

$$P(W|R, S) = 0.95$$

$$P(W|R, \neg S) = 0.80$$

$$P(W|\neg R, S) = 0.70$$

$$P(W|\neg R, \neg S) = 0.05$$

Step 1: Use law of total probability

$$\begin{aligned} P(W) = & P(W|R, S)P(R)P(S|R) + P(W|R, \neg S)P(R)P(\neg S|R) \\ & + P(W|\neg R, S)P(\neg R)P(S|\neg R) + P(W|\neg R, \neg S)P(\neg R)P(\neg S|\neg R) \end{aligned}$$

Step 2: Compute each term

1. Rain & Sprinkler On: $0.95 * 0.3 * 0.01 = 0.00285$

2. Rain & Sprinkler Off: $0.80 * 0.3 * 0.99 = 0.2376$

3. No Rain & Sprinkler On: $0.70 * 0.7 * 0.40 = 0.196$

4. No Rain & Sprinkler Off: $0.05 * 0.7 * 0.60 = 0.021$

Step 3: Add all terms

$$P(W) = 0.00285 + 0.2376 + 0.196 + 0.021 = 0.45745 \approx 0.457$$

Result:

Probability that the grass is wet, $P(W) \approx 0.457$ (45.7%)